Test Specification: Enterprise Search Feature

End-to-End Functional, Negative, Security & Performance Test Suite

Author: Senior QA Automation Engineer
Application: Enterprise E-Commerce Platform
Execution: Automated (Playwright / Cypress) & Manual

TOTAL TEST CASES	P0 / BLOCKERS	P1 / CRITICAL	P2 / SECONDARY	TARGET SLA
15 Scenarios	6 Cases	6 Cases	3 Cases	< 500ms TTFB

MODULE 1: FUNCTIONAL SEARCH, FILTERING & SORTING

TEST ID	SCENARIO	TEST STEPS	TEST DATA	EXPECTED RESULT	PRIORITY
TC-SRC-001	**Exact Keyword Search**	1. Focus on global search bar. 2. Type product name. 3. Click Search button or press Enter.	`"wireless noise cancelling headphones"`	Search results page loads displaying relevant products. Exact matching items appear at the top with matching tokens highlighted.	P0
TC-SRC-002	**Fuzzy & Typo Tolerance Search**	1. Input misspelled keyword. 2. Execute search. 3. Verify "Did you mean?" banner and auto-corrected results.	`"hedphones", "samusng"`	Shows corrected results with prompt: *"Showing results for headphones. Search instead for hedphones"*.	P1
TC-SRC-003	**Multi-Faceted Category Filter**	1. Search for broad category term. 2. Select Brand and Price Range facet checkboxes. 3. Verify URL parameter update and product list refresh.	`Keyword: "laptop" Brand: Apple Price: $1000 - $2000`	Result set refines asynchronously without full-page reload. Result counter updates accurately. Selected chips displayed with 'Clear' option.	P0
TC-SRC-004	**Sorting Criteria Application**	1. Execute search query. 2. Open "Sort By" dropdown. 3. Select "Price: High to Low", then "Customer Rating".	`Query: "running shoes" Sort: Price: High to Low`	Products re-ordered in strict descending numerical order based on active selling price. Filter selections remain preserved.	P1
TC-SRC-005	**Pagination & State Persistence**	1. Execute search returning >50 items. 2. Navigate to Page 2. 3. Click a product, then trigger browser back button.	`Query: "cotton shirt" Page: 2`	Page 2 items render. Browser back navigation preserves query, scroll position, page index, and applied facet states.	P1

Automation Precondition: ElasticSearch index must be synchronized with the master product catalog database prior to running test execution cycles. Ensure test fixtures include varied inventory statuses (In-Stock, Out-of-Stock, Discontinued).

MODULE 2: NEGATIVE, SECURITY & BOUNDARY SCENARIOS

Evaluates the resilience, sanitization layer, and error-handling capabilities of the search endpoint against malicious inputs, unexpected characters, and boundary payload limits.

TEST ID	SCENARIO	TEST STEPS	TEST DATA	EXPECTED RESULT	PRIORITY
TC-SEC-001	**SQL Injection (SQLi) Prevention**	1. Enter SQL injection payload into search input. 2. Submit search via UI and direct GET/POST query parameter. 3. Inspect backend database logs and response code.	`' OR '1'='1' --` `" UNION SELECT null, username, password FROM users --`	Input is fully escaped and treated as raw string literal. Zero DB error disclosure; returns HTTP 200 with standard "No results found" view or 400 Bad Request.	P0
TC-SEC-002	**Cross-Site Scripting (XSS) Mitigation**	1. Input JavaScript/HTML payload into search bar. 2. Execute search. 3. Verify search breadcrumb and title render in DOM.	`<script>alert(document.cookie)</script>` `<img src=x onerror=alert(1)>`	Payload is HTML entity-encoded (`<script>`). Script does NOT execute in browser context. CSP headers block inline execution.	P0
TC-NEG-003	**Empty Input & Whitespace Handling**	1. Leave search bar blank and press Enter / click icon. 2. Enter multiple space/tab characters and submit.	`"" (empty string)` `" \t " (whitespace)`	Submission prevented on client-side or redirects cleanly to home/catalog root. No unhandled null pointer or infinite spinner triggered.	P1
TC-NEG-004	**Excessive Payload Boundary Test**	1. Paste a large string exceeding maximum character limits. 2. Submit form. 3. Validate client-side truncation and server response.	`String with 2,048+ alphanumeric characters`	Input is truncated at max limit (e.g., 255 chars) with friendly validation notice. Backend gracefully handles request without 500 Internal Server Error or memory spike.	P1
TC-NEG-005	**Special Characters & Regex Metachars**	1. Enter search string containing reserved Lucene / regex operators. 2. Execute search.	`+ - && || ! ( ) { } [ ] ^ " ~ * ? : \ / @ # $ % ^ & *`	Search engine escapes parser characters without returning index query syntax exceptions. Displays clear empty state with helpful recommendations.	P1

Security Baseline Note: Automated security regression tests must run against DAST pipelines (OWASP ZAP / Burp Suite) before production releases. Ensure Web Application Firewall (WAF) rule sets allow compliant parameter scanning without false dropouts.

MODULE 3: PERFORMANCE, USABILITY & UI SCENARIOS

Validates end-user response times, dynamic predictive auto-suggestions, asynchronous debounce intervals, and responsive rendering across varied viewports.

TEST ID	SCENARIO	TEST STEPS	TEST DATA	EXPECTED RESULT	PRIORITY
TC-PRF-001	**Auto-Suggestions Latency & Debounce**	1. Type characters into search bar at rapid cadence (>100 WPM). 2. Verify network panel request throttling. 3. Inspect dropdown suggestion render time.	`Debounce threshold: 300ms Input: "wirel"`	API triggers exactly 300ms after user pauses typing. Dropdown displays within 200ms of API response. Matching characters highlighted.	P0
TC-PRF-002	**Search Response Time Under Load**	1. Execute search request during simulated load (500 virtual concurrent users). 2. Measure Time to First Byte (TTFB) and Total Page Load Time.	`Concurrent VUs: 500 Target SLA: < 500ms TTFB`	95th percentile response time remains <500ms; full UI render with product cards completes within <1.5s without gateway 504 timeouts.	P1
TC-UI-003	**Keyboard Navigation & Accessibility (a11y)**	1. Navigate to search bar using `Tab`. 2. Type query and navigate suggestions using `Arrow Down/Up`. 3. Press `Enter` on highlighted recommendation.	`Key events: Tab, ArrowDown, Escape, Enter`	Focus state clearly visible (WCAG 2.1 AA compliant). `Escape` closes dropdown; `Enter` redirects to selected suggestion URL.	P1
TC-UI-004	**Mobile & Small Viewport Search UX**	1. Emulate mobile viewport (375x812 - iPhone 13/14). 2. Tap search icon to trigger search overlay. 3. Execute query and observe filter drawer.	`Viewports: 375px, 414px, 768px`	Search opens full-screen modal with sticky top bar and auto-focused keyboard. Filters open in slide-out drawer with floating 'Apply' button.	P2
TC-PRF-005	**Search Query Result Caching**	1. Execute popular search query. 2. Record response time and cache header. 3. Repeat identical search within 60 seconds.	`Header inspect: X-Cache: HIT Query: "iphone"`	Second search serves from Edge CDN/Redis cache (`X-Cache: HIT`) with response time <80ms, offloading database compute.	P2

Automation Implementation Strategy

Framework: Playwright TypeScript with Page Object Model (POM).
CI/CD Integration: P0/P1 regression tests run on every pull request via GitHub Actions. Performance tests execute nightly with k6.
Test Data Fixtures: Deterministic ElasticSearch test indices hydrated via Docker container before suite execution.